

Mid-Semester Examination Answer Sheet
CS633: Parallel Computing **Date: 28th February 2026** **Time: 8 AM – 10 AM**

Roll number 230316

Name Chaitanya Vishnoi

(0 marks will be awarded if name and roll numbers are left empty)

PLEASE SWITCH OFF YOUR MOBILE PHONE AND KEEP IT IN YOUR BAG

- Write your name and roll number, your paper will not be evaluated if these are missing.
- Total time is 120 minutes. Total marks = 100.
- All questions are compulsory. There is no partial marking in programming questions.
- Write your name and roll number on rough sheets as well.
- You **must** answer all questions **on this sheet only** in the designated space. Violations lead to a penalty of -5. Ample space has been provided for every question. Think and be sure before you write your final answer on the answer sheet because question paper and extra sheets will not be collected for evaluation.
- **Legible** and neat answers within the designated area will be graded only; no marks will be awarded for illegible text.
- The MPI function prototypes are given on the last page of the question paper and the answer sheet.
- Read the questions carefully before answering. All clarifications must be directed to the instructor only.
- Extra points will not be given for being verbose, so answer to the point, without being abrupt or confusing.
- Please do not leave the exam hall before we finish counting.
- **DO NOT OPEN THE QUESTION PAPER BEFORE THE SCHEDULED TIME. IF YOU ARE FOUND DOING SO, 5 MARKS WILL BE DEDUCTED AND YOU WILL LOSE 5 MINUTES.**

Marks

1a	1b	1c	1d	2a	2b	2c	3	4a	4b	5a	5b
1	2	4	4	4	4	7	4	5	3+4	4	3
6a	6b	6c	7a	7b	8a	8b	9a	9b	10a	10b	Total
3	4	4	5	5	4	5	6	5	5	5	100

I pledge to answer all questions by myself without taking any external help.

Chaitanya (Sign your name)

A1. Write TRUE or FALSE (do not write T or F) (10 Marks)

(a). A new system, called SMILE, in the CSE department of IIT Kanpur has 40 nodes. Each node has 10 cores. This means that a job executed on SMILE via the command `mpiexec -np 500 -f hostfile ./executable` will fail, because of fewer available resources. Assume *hostfile* has the names of all the nodes of SMILE.

False

(b). If a user submits an MPI job to the above-mentioned system, SMILE, the user must first install a job scheduler on SMILE, otherwise the user's job will not run on SMILE.

True

(c). The above-mentioned system, SMILE, has been designed to run at a peak throughput of 10 GFLOPS (floating point operations per second). This means that

(i) A job that runs on SMILE will take a maximum of 10 ns to run False

(ii) Any job that runs on SMILE must run at 10 GFLOPS True

(iii) No job will run at more than 10 GFLOPS True

(iv) It is possible that a code that runs on SMILE and gives a throughput of less than 10 GFLOPS cannot be optimized further True

(d) If a user submits a job on a supercomputer SMILE++ for a wall-clock time of 20 minutes, and the actual runtime of the user's job is 30 minutes, this means that

(i) The job will be terminated by the system administrator False

(ii) The job will be terminated by the scheduler True

(iii) The job will be in an idle state unless the user kills the job False

(iv) The user's job will run for 30 minutes; however, 10 minutes will be deducted from the next job of the user True

A2. 4 + 4 + 7 Marks

(a) Derive the number of hops for sender ranks 0 and 6, for their respective D1 and D2 communications.

Rank 0: D1 0

Rank 0: D2 0

Rank 6: D1 0

Rank 6: D2 0

(b) Derive the maximum available bandwidth for sender ranks 0 and 6, for their respective D1 and D2 communications.

Rank 0: D1 _____

Rank 0: D2 _____

Rank 6: D1 _____

Rank 6: D2 _____

(c) Derive the total number of outgoing communications to higher levels (i.e. outward to spine switches) along with the communication volumes from L1 and L2 leaf switches for the given execution configuration for sending to rank+D1 and rank+D2 ranks only.

L1 _____

L2 _____

A3. Marks: 4 (No partial marks, use as many lines below as required for your answer)

Write the output corresponding to line 14 before writing the output corresponding to line 22 for each rank (sorted order of ranks for both cases), otherwise, penalty of -2.

0 before 17 27

1 before 17 27

2 before 17 27

3 before 17 27

0 after 17 30

0 after 17 30

0 after 7 30

0 after 17 30

A4-a. Marks: 5 (No partial marks, use as many rows and columns as required for the output, leave the remaining boxes empty)

Write the output in sorted order of the first column, otherwise, penalty of -2.

0	0	1	0		
1	0	0	1		
2	4	4	0		
3	1	1	0		
4	1	0	1		
5	4	0	1		
6	2	1	0		
7	2	0	1		
8	4	0	2		

Example valid answer

a	b	
c	d	

Example invalid answer

	a	b
c		d

A4-b. Marks: 3 + 4

For the output command `mpirun -np 9 -hosts host1,host2,host3 ./exe 3 | sort -k1n`

(i) Mention the rank placement on the three hosts and (ii) For a naïve many-to-one implementation of MPI_Gather (line 27), derive the average number of hops, assuming the hosts connectivity shown on the question paper.

(i) host1 0, 3, 6
 host2 1, 4, 7
 host3 2, 5, 8

(ii) Write in this format: "Hops from ____ to ____ = ____". Assume intra-node hop count = 0.

Hops from 0 to 1 = 1
 Hops from 0 to 4 = 1
 Hops from 0 to 7 = 1
 Hops from 0 to 2 = 2
 Hops from 0 to 5 = 2
 Hops from 0 to 8 = 2
 average = $\frac{1+1+1+2+2+2}{6}$
 $\therefore \text{avg} = 1$
~~average = $\frac{1+1+1+2+2+2}{6} = \frac{9}{6} = 1.5$~~

A5. Marks: 4+3 (Use as many lines as required for the answer)

For the output command `mpirun -np 84 ./exe | sort -k1n`

(a) Write the output corresponding to line 18 (sorted order of first column)

0 New 0 Max 21
 1 New 0 Max 21
 2 New 0 Max 21
 3 New 0 Max 21

(b) Write the output corresponding to line 21 (sorted order of first column)

0 Max 84

A6-a. Assume 32-core hosts. Match the following executions with the times (average of 3 runs) shown on the boxes. (3 Marks)

- (A) `time mpirun -np 60 --hosts host1,host2,host3,host4 ./3 100`
 (B) `time mpirun -np 60 --hosts host1,host2,host3,host4 ./3 10000000`
 (C) `time mpirun -np 120 --hosts host1,host2,host3,host4 ./3 10000000`

2.8 s	1.4 s	0.2 s
C	B	A

A6-b. Assume 32-core hosts. Which among (i) and (ii) takes longer time to execute and why. Explain your answer. (4 Marks)

- (i) `mpirun -np 120 --hosts host1,host2,host3,host4 ./3 6000000`
 (ii) `mpirun -np 120 --hosts host1:30,host2:30,host3:30,host4:30 ./3 6000000`

(ii) takes longer to execute
 This is because, the 30 elements which host 4 contain are bigger in value than those in host 1 on average. However, in (i) the average is relatively similar across all hosts. Hence, host 4 increases execution time for (ii)
 host 4 (w.r.t. host 1)

A6-c. Assume 12-core hosts. Execution command is `mpirun -np 24 --hosts host1:6,host2:6,host3:6,host4:6 ./exe 1024`

Assume that host1 – host4 are connected using a linear array, i.e. number of hops from host1 to host4 = 3. Assume that the above execution took 0.1 seconds. Derive the communication bandwidth achieved by the code in this case. (4 Marks)

A7-a. Assume 40-core hosts. Fill the following execution number (A and B) on two correct circles only. Unclear answers such as illegible text and filling more than 2 circles will lead to 0 marks. (5 Marks)

(A) mpirun -np 120 -hosts host1:40,host2:40,host3:40 ./5 600 4
(B) mpirun -np 120 -hosts host1:40,host2:40,host3:40 ./5 600 16

0.050770 0.050770

A

0.050770 0.060870

0.070970 0.050770

0.109982 0.109982

0.109947 0.124082

B

0.129088 0.109911

A7-b. Below lines show the timings (only max_time1) for the corresponding executions (average of 3 runs). Assume that there are no other jobs running on the system with these four hosts. Assume 32-core hosts.

mpirun -np 32 -hosts host1:8,host2:8,host3:8,host4:8 ./exe 6000000 16
max_time1 -> 0.0035

mpirun -np 128 -hosts host1:32,host2:32,host3:32,host4:32 ./exe 6000000 16
max_time1 -> 0.0085

Explain in detail (write complete but short sentences) two main reasons for the difference in timings. (5 Marks)

1. ~~The~~ Firstly, the difference in total tasks is in the ratio of 1:4 for the two cases. Whereas, for second execution, the ppn and total process both increase 4-fold.
2. The other differentiating factor could be the work they are doing. Find the a random number among a bigger sample for larger number of iterations.

A8-a. Replace the below two lines in the given code with a single line of code to essentially achieve the same intent. Write that line below. You may use **recvdata** declared in line 14 in your new line of code. Note that the modified code (i.e. the one with your new line of code will result in the same output from lines 25 – 31 just by replacing **data** with **recvdata** in line 28. (4 Marks)

```
22 for (int i=0; i<size; i++)
23   MPI_Bcast (&data[i][0], count, MPI_INT, i, MPI_COMM_WORLD);
```

MPI_Algorithm

A8-b. Write the output of the execution command: **mpirun -np 3 ./exe 3**

Use as many rows and columns as required for the output, leave the remaining boxes empty. Write the output in sorted order of the first column, otherwise, penalty of -2. (5 Marks) No partial marks.

0	3				
1	5				
2	13				
3	3				
4	5				
5	4				
6	2				
7	3				
8	6				

Example valid answer

C	K	
B	Y	

Example invalid answer

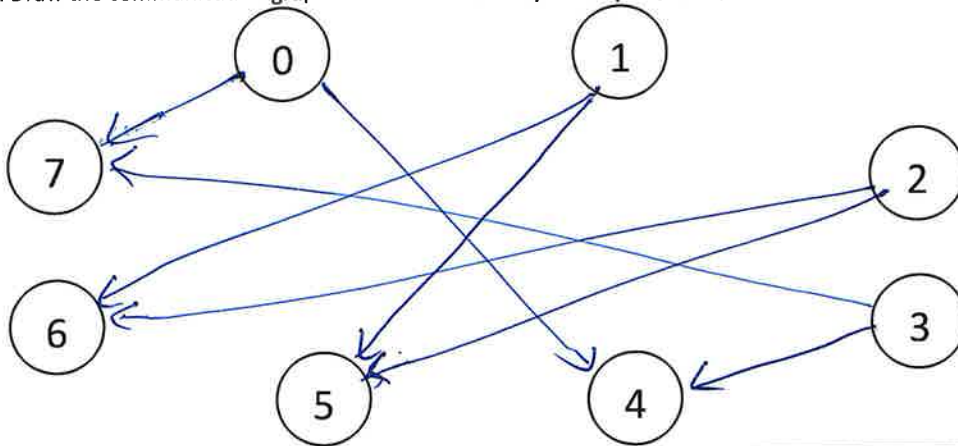
		C	K
B			Y

A9-a. What is the output of the command **mpirun -np 4 ./exe | sort -k1n**

Use as many rows and columns as required for the output, leave the remaining boxes empty. Write the output in sorted order of the first column, otherwise, penalty of -2. See examples of valid answers in 4a and 8a. (6 Marks)

0	2	3	0	0	
1	3	2	1	2	
2	0	1	3	5	
3	1	0	4	7	

A9-b. Draw the communication graph for the execution `mpirun -np 8 ./exe` (5 Marks)



A10-a. (5 Marks)

$$\frac{N}{P} \text{ processes lead to } \frac{4\sqrt{P}}{\sqrt{N}}$$

$$\therefore \frac{4\sqrt{P}}{(N/P)} \rightarrow \frac{4\sqrt{P}}{\sqrt{N}}$$

A10-b. (5 Marks)

$$\frac{4\sqrt{N}}{\sqrt{P}}, \quad \frac{4\sqrt{N}}{N/P} = \frac{4\sqrt{P}}{\sqrt{N}}$$

MPI FUNCTIONS (PROTOTYPES)

Size of integer = 4 bytes, 1 KB = 1024 bytes, 1 MB = 1024 KB

- 1) int **MPI_Comm_rank** (MPI_Comm comm, int *rank)
- 2) int **MPI_Comm_size** (MPI_Comm comm, int *size)
- 3) int **MPI_Get_processor_name**(char *name, int *resultlen)
- 4) int **MPI_Comm_split**(MPI_Comm comm, int color, int key, MPI_Comm * newcomm)
- 5) int **MPI_Group_incl**(MPI_Group group, int n, const int ranks[], MPI_Group *newgroup)
- 6) int **MPI_Comm_create_group**(MPI_Comm comm, MPI_Group group, int tag, MPI_Comm * newcomm)
- 7) int **MPI_Comm_group**(MPI_Comm comm, MPI_Group * group)
- 8) int **MPI_Send** (const void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)
- 9) int **MPI_Recv** (void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)
- 10) int **MPI_Get_count** (const MPI_Status *status, MPI_Datatype datatype, int *count)
- 11) double **MPI_Wtime**(void)
- 12) int **MPI_Bcast**(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm)
- 13) int **MPI_Gather**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm)
- 14) int **MPI_Scatter**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm)
- 15) int **MPI_Reduce**(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
- 16) int **MPI_Allreduce**(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
- 17) int **MPI_Allgather**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, MPI_Comm comm)
- 18) int **MPI_Alltoall**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, MPI_Comm comm)
- 19) int **MPI_Pack**(const void *inbuf, int incout, MPI_Datatype datatype, void *outbuf, int outsize, int *position, MPI_Comm comm)
- 20) int **MPI_Unpack**(const void *inbuf, int insize, int *position, void *outbuf, int outcount, MPI_Datatype datatype, MPI_Comm comm)